

File Upload Vulnerabilities

From Simple Bugs to Remote Code Execution (RCE)

Understanding Risk Escalation in Web Applications



#WebSecurity

#Pentesting

#CTF

#RCE

#FileUpload

What Is File Upload Functionality?

Common Use Cases



Profile Pictures

Avatar/photo uploads on social platforms



Document Uploads

PDFs, reports, resumes in web portals



Email Attachments

Files shared via webmail interfaces



Media Content

Videos/audio for streaming apps



Software Packages

Plugin or module uploads in CMS

Why It Increases Attack Surface



Untrusted files enter the server environment



Filenames may contain path traversal sequences



MIME types are easily spoofed by attackers



Executable files can masquerade as images



Storage paths may be web-accessible



Insufficient validation at client or server side

How File Upload Handling Works



CLIENT-SIDE VALIDATION

HTML accept= attribute restricts types

JavaScript validates before submission

File size checks in the browser



Trivially bypassed by an attacker



Never rely on it as sole protection



SERVER-SIDE PROCESSING

Extension whitelist / blacklist checks

MIME type detection (Content-Type)

Magic byte / file signature analysis

File renamed and stored on disk

Web server decides if file is executable

Key Concepts: **MIME Type** = metadata describing file format (e.g. image/png) | **Extension** = suffix like .php, .jpg | **Storage Path** = where file lands on the server filesystem

Common File Upload Validation Failures

✗ Extension-Only Checks

Server only checks file extension (.jpg, .png). Attacker renames shell.php to shell.jpg — check passes.

✗ Weak Content-Type Verification

Server trusts Content-Type header sent by the browser. Attacker crafts request with Content-Type: image/jpeg but uploads PHP file.

✗ Client-Side Validation Trust

Browser-based JS validation is bypassable. Attacker uses Burp Suite to intercept and replace payload post-validation.

✗ Executable Upload Directories

Web server is configured to execute scripts in the upload folder. Any uploaded .php file can be triggered via a direct URL request.

File Upload → Stored XSS

How It Works

- Attacker uploads a malicious SVG or HTML file
- Server stores it in a web-accessible path
- Victim browses to or views the uploaded file
- Browser executes embedded JavaScript
- Cookies, tokens, or keystrokes are stolen

Educational Payload Concepts

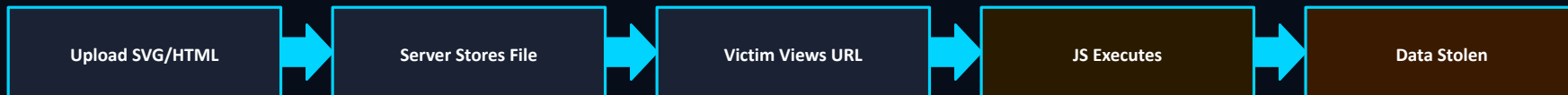
SVG – Script Execution via XML

```
<svg xmlns="http://www.w3.org/2000/svg">  
  <script>  
    // Script executes when SVG is opened  
    // in a browser context  
  </script>  
</svg>
```

HTML File Upload – Browser Execution

```
<!-- Uploaded .html file served by server -->  
<html><body>  
  <script>/* runs in same origin */</script>  
</body></html>
```

Attack Flow



File Upload → HTML Injection

What Happens

Attacker uploads a crafted .html or .htm file

Server stores it without restricting HTML execution

File is served directly by the web server

Browser renders it in the site's origin context

Injected forms, iframes, or scripts execute

Same-Origin Risks

- Reads cookies and local storage of the same domain
- Performs credentialed AJAX requests to the application
- Phishing forms injected into the legitimate domain
- Can redirect users to attacker-controlled pages

Difference from XSS

HTML Injection via upload doesn't require reflected/stored XSS in the app code — the file itself becomes the malicious page, served under the app's trusted origin.

File Upload → Information Disclosure

Public Upload Directories

Files stored in /uploads/ that are web-accessible allow direct URL enumeration. Attackers guess paths to access other users' files.

```
https://target.com/uploads/resume.pdf
```

Directory Listing Exposure

Misconfigured Apache/Nginx serves a listing of all uploaded files. Attacker browses the entire contents of the upload folder.

```
Index of /uploads/ [all files visible]
```

File Overwrite Scenarios

Predictable filenames allow overwriting existing files. An attacker can replace legitimate content or probe for file existence via status codes.

```
PUT /uploads/config.txt → overwrites  
original
```

File Upload → Path Traversal

Filename Manipulation Concept

When a server uses a user-supplied filename directly in a file path without proper sanitization, an attacker can craft filenames containing directory traversal sequences.

These sequences instruct the operating system to navigate up the directory tree, potentially writing files outside the intended upload directory.

Traversal Pattern Example

Filename in request:

```
../../../../etc/passwd
```

Decoded path on server:

```
/var/www/uploads/  
../../../../etc/passwd
```

Resolved to:

```
/etc/passwd
```

How Poor Sanitization Causes Traversal

- Server strips only single-level '../' — double-encoding or nested sequences bypass the filter
- Null-byte injection (%00) terminates extension check while OS sees full malicious path
- Unicode/UTF-8 encoding of '/' or '..' passes pattern matching but decodes on disk

What Is Remote Code Execution (RCE)?

Definition: Remote Code Execution (RCE) is a vulnerability class that allows an attacker to run arbitrary code on a target system from a remote location — without physical access or an existing account on the server.

Why RCE Is Considered Critical

Ranked CVSS 9.0-10.0 — highest severity class

No prior access or credentials required

Attacker gains control at the process privilege level

Enables all downstream attacks (data theft, ransomware)

Single vulnerability can compromise entire infrastructure

Impact: Full System Compromise

 Data Exfiltration — All database and file contents

 Credential Theft — Passwords, keys, tokens

 Lateral Movement — Pivot to internal systems

 Ransomware Staging — Encrypt, destroy, extort

 Privilege Escalation — Root / SYSTEM access

How File Uploads Can Lead to RCE

Uploading Server-Interpreted Files

Web servers like Apache/Nginx with PHP or JSP modules will execute uploaded scripts when requested. A .php file in /uploads/ becomes executable code.

```
shell.php →  
/var/www/html/uploads/  
shell.php  
GET  
/uploads/shell.php  
→ PHP executes!
```

Double Extensions

Filename like image.php.jpg passes extension checks (ends in .jpg) but Apache may still pass it to PHP handler depending on server config.

```
filename:  
image.php.jpg  
Apache AddHandler:  
*.php → executes  
PHP
```

MIME Type Confusion

Server checks Content-Type: image/jpeg but doesn't verify actual file content. Actual PHP code slips through as a 'jpeg'.

```
Content-Type:  
image/jpeg  
[Actual content:  
<?php system(...);  
?>]
```

Misconfigured Web Servers

Upload directories with permissive execution settings, .htaccess overrides allowed, or mod_php enabled for all directories.

```
.htaccess: AddType  
application/x-httpd-  
php .jpg  
→ All .jpg files  
now execute as PHP
```

File Upload → Web Shell

Web Shell: A script uploaded to a vulnerable web server that, once accessible via browser, provides an attacker with remote command-line control of the underlying OS through the web interface.

Key Characteristics

- Typically a single small script file (PHP, JSP, ASPX)
- Accepts OS commands via URL parameter or form input
- Returns command output in the HTTP response
- Provides persistent access — survives server restarts
- Used for reconnaissance, exfiltration, and pivoting

Conceptual Execution Model

```
# Conceptual flow only

1. Attacker uploads script file
   └─ File stored on web server disk

2. Attacker sends HTTP request
   └─ URL: /uploads/shell.ext?cmd=...

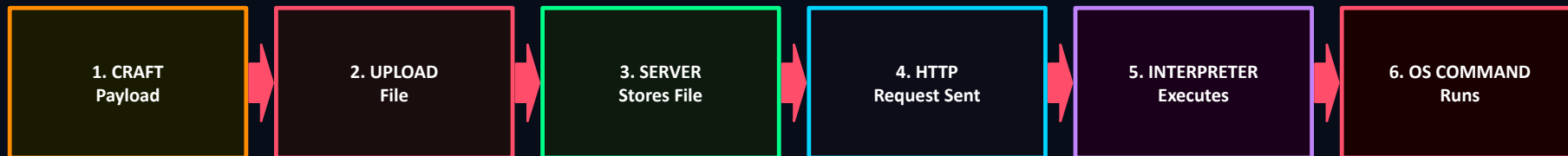
3. Web interpreter receives request
   └─ Executes embedded OS command

4. Command output returned in response
   └─ Attacker sees server internals
```

📌 *Note: These are conceptual learning examples only. Uploading web shells to systems without explicit authorization is illegal and unethical. Always use authorized lab environments for testing.*

RCE Execution Flow

Upload → Storage → Interpretation → System Compromise



PHASE 1 – Preparation & Upload

- ▶ Attacker crafts executable script with OS command passthrough
- ▶ Bypasses client-side validation (modified request headers)
- ▶ Spoofs MIME type to pass server-side checks
- ▶ File arrives on server disk in upload directory

PHASE 2 – Activation

- ▶ Attacker sends HTTP GET/POST to uploaded file URL
- ▶ Web server routes request to script interpreter (PHP/JSP)
- ▶ Interpreter processes the script and executes OS calls
- ▶ Commands run under the web server process privilege

PHASE 3 – Post-Exploitation

- ▶ Command output returned in HTTP response body
- ▶ Attacker reads file system, network config, credentials
- ▶ Establishes persistent reverse shell or backdoor
- ▶ Pivots to internal network or escalates privileges

Post-RCE Impact

What happens after an attacker achieves code execution

Credential Access

- ▶ Read /etc/passwd, /etc/shadow (Linux)
- ▶ Dump database credentials from config files
- ▶ Extract API keys and secrets from environment variables
- ▶ Access .ssh/ folders and private keys

Data Exfiltration

- ▶ Export entire database contents
- ▶ Compress and send sensitive files to attacker's server
- ▶ Access PII, financial records, intellectual property
- ▶ Exfiltrate user credentials for credential stuffing

Lateral Movement

- ▶ Scan internal network from compromised server
- ▶ Use stolen credentials to access adjacent systems
- ▶ Exploit trust relationships between internal services
- ▶ Attack internal applications not exposed to internet

Privilege Escalation

- ▶ Run local exploits to gain root/SYSTEM access
- ▶ Abuse SUID binaries or misconfigured sudo rules
- ▶ Install persistent backdoors and rootkits
- ▶ Disable security logging and monitoring

Real-World RCE Case Examples

High-level breaches caused by insecure file uploads

2017

ImageMagick (ImageTragick)

A vulnerability in the ImageMagick image-processing library allowed attackers to execute OS commands by uploading a specially crafted image file. Thousands of sites using image upload functionality were affected.

CVE-2016-3714

2019

WordPress File Manager Plugin

A critical RCE flaw in the popular WordPress File Manager plugin allowed unauthenticated attackers to upload malicious PHP files directly through the plugin's file management interface.

CVE-2020-25213

2021

Accellion FTA Breach

Attackers exploited an RCE vulnerability via file upload in Accellion's legacy File Transfer Appliance. Sensitive data from dozens of organizations including government agencies was exfiltrated.

CVE-2021-27101

2022

MOVEit Transfer (SQLi + Upload Chain)

Exploited vulnerabilities in MOVEit Transfer's file upload handling allowed attackers to deploy web shells and execute commands, affecting hundreds of organizations globally in a massive data theft campaign.

CVE-2023-34362

Preventing File Upload Vulnerabilities

✓ Allow-List Extension Validation

Only permit a predefined set of safe extensions (e.g., .jpg, .png, .pdf). Reject all others. Never use a blacklist — attackers find edge cases.

🎲 Randomized Filenames

Generate a cryptographically random UUID or hash as the stored filename. Never use the attacker-controlled original filename on disk.

🚫 Non-Executable Storage

Store uploaded files outside the web root (e.g., /var/uploads/) or on a separate CDN/storage service. Files should never be directly executable.

🔬 File Type Verification

Validate file content using magic bytes / file signatures — not just extensions or Content-Type headers. Use libraries like libmagic or python-magic.

Preventing RCE Specifically

01

Disable Script Execution in Upload Folders

Configure web server (Apache, Nginx) to never execute scripts in upload directories. Use Options -ExecCGI and remove PHP/JSP handlers from upload paths.

```
# Nginx: deny script execution
location /uploads/ {
    add_header Content-Type text/plain;
}
```

02

Least Privilege for Web Server Process

Run the web server as a restricted, unprivileged user. Even if RCE occurs, the attacker's access is limited to what that process can do — preventing system-wide compromise.

```
# Run web server as 'www-data'
# No shell, no home dir, no sudo
```

03

Proper Server Configuration

Disable .htaccess overrides in upload directories. Restrict PHP/interpreter configuration to trusted directories only. Use security-focused web server config hardening guides.

```
AllowOverride None
PhpAdmin_value engine Off
```

04

Logging and Monitoring

Log all file upload attempts with IP, filename, and MIME type. Alert on suspicious file types. Monitor for file creation events in upload directories using auditd or SIEM.

```
# Alert on: .php .jsp .asp .sh
# Monitor: /var/www/uploads/
```


Defense-in-Depth Strategy

Layered controls — one bypass doesn't mean full compromise

OUTER: Web Application Firewall (WAF)

Blocks known malicious upload patterns, oversized files, and suspicious filenames at the network perimeter before reaching the app.

LAYER 2: Antivirus / Malware Scanning

Scan uploaded files with AV engines before storage. Integrate ClamAV or cloud-based scanning APIs for known malware signatures.

LAYER 3: Content Security Policy (CSP)

Browser-level control preventing inline scripts and unauthorized origins. Even if XSS is achieved via upload, CSP limits its impact.

INNER: Sandboxing & Isolation

Process uploaded files in an isolated container or VM. Any malicious payload is contained — it cannot reach the host OS or network.

✓ *Defense-in-depth means: multiple independent controls so that bypassing one layer does not result in system compromise.*

Summary & Key Takeaways

01

File upload bugs escalate gradually

A misconfiguration that allows HTML uploads today can become RCE when server config changes. Treat every upload vector seriously.

02

Early validation prevents critical compromise

Enforce strict, server-side allow-list validation at the point of upload. Client-side checks are cosmetic — not security controls.

03

Secure design is essential — not optional

Architecture decisions (storage outside web root, randomized names, separated execution) prevent entire vulnerability classes from existing.

04

RCE = Full Compromise — treat it as existential

An RCE vulnerability via file upload is a CVSS 9.0-10.0 finding. Prioritize elimination through strict upload controls and server hardening.



Practice in authorized CTF environments and bug bounty programs. Never test on systems without explicit written permission.